

We'll have more choices.

数据类型



内容要点

- 常量和变量
- 常数的表示
- 数据类型
 - 整型
 - 浮点型
- 类型的选用
- 常见的编程错误

目录

	1	变量和常量
	2	常数的表示
	3	整型和浮点型
	4	相关的编程错误
	5	小结

```
/* platinum.c -- your weight in platinum */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

新的数据类型：浮点型

```
float weight; /* user weight */
```

```
float value; /* platinum equivalent */
```

```
printf("Are you worth your weight in platinum?\n");
```

```
printf("Let's check it out.\n");
```

```
printf("Please enter your weight in pounds: ");
```

对普通变量，此处需要&号

```
/* get input from the user */
```

```
scanf("%f", &weight);
```

此处程序暂停，等待用户键盘输入，直到按下回车键（Enter）继续

```
/* assume platinum is $1700 per ounce */
```

```
/* 14.5833 converts pounds to ounces */
```

表示常数的新方法：带小数点表示浮点型

```
value = 1700.0 * weight * 14.5833;
```

一个程序应该有0个、1个或者多个输入

```
printf("Your weight in platinum is worth $%.2f.\n",  
value);  
printf("You are easily worth that! If platinum prices  
drop,\n");  
printf("eat more to maintain your value.\n");  
return 0;  
}
```

打印常数的新方法: printf()中的
%f用于处理浮点型, %.2f只输出2位
有效小数, 更加美观。

Are you worth your weight in platinum?
Let's check it out.

Please enter your weight in pounds: 156↓

Your weight in platinum is worth \$3867491.25.

You are easily worth that! If platinum prices drop,
eat more to maintain your value.

蓝色下划线表示键盘输入,
↓ 表示按下回车键

数学和计算机中的变量

- 数学和物理学中的变量（variable）

- 表达式或公式中，没有固定的值而可以变动的数或量

随自变量变化的量 $y = f(x)$ 自变量，自由变化的量

- 计算机中的变量

- 抽象的存储地址，它含有

- 值：某种已知或未知的信息量
- 名称：配对了关联的符号名称

```
int x = 20;
```

- 通常使用变量名称引用存储值

变量的属性

- 计算机中的变量是抽象的存储地址 `int weight = 20;`

类别	属性	值（示例）
基本	名称	weight
	值	20
	物理意义	重量
	数据类型	整型
内存相关	地址	0x3000~0x3003
	长度	4字节
	电平表示	十六进制： 0x14, 0x00, 0x00, 0x00 二 进 制： 0010 1000 000.....00

变量的操作

• 变量的操作

操作	作用	示例
声明	将名称与存储空间对应	<code>int weight;</code>
写入值	改变变量的值	<code>weight = 20;</code>
读取值	读取变量的值	<code>max_weight = weight * 1.2;</code>

• 注意事项

- 变量必须先声明后使用，先赋值再求值
- 初学者应养成声明时赋值的好习惯

变量的声明

• 变量声明的格式

分类	格式	示例
声明一个变量	<类型> <变量名>;	<code>int var;</code>
声明多个变量	<类型> <变量名>, <变量名2>;	<code>int grade, class, x;</code>
声明时赋值	<类型> <变量名> = <表达式>;	<code>int grade, class = 2;</code>

• 变量声明的作用

Times字体的尖括号内容可选但不可省略

操作	说明	示例
开辟存储空间	在栈区找到并占用空间	<code>int v = 2;</code> 0x3000~0x3003
指定空间格式	数据类型指明空间的尺寸、电平格式	4字节, 整型
空间关联名称	将指定名称与存储空间对应	将v关联到空间
声明时赋值	如有, 用该值改变该存储空间的值	赋值改空间为2

变量的声明

- 允许声明语句的位置
 - 在所有函数之外
 - 在代码块内
 - 在for的第一个分量
 - 在参量列表
- 必须在使用前声明
 - 早期限定在语句块最开头集中声明
 - C99起放开

```
#include <stdio.h>
int a = 3;
int print(int v) {
    printf("%d\n", v);
}
int main() {
    int x = a + 2;
    for (int i = 0; i < 3; i++) {
        print(x - i);
    }
    return 0;
}
```

在所有函数之外

在参量列表

在代码块内

在for的第一个分量

变量的命名

- 变量命名应符合一般命名规则
 - 字符集：小写字母、大写字母、数字、下划线（_）
 - 不可与保留字一致
 - 不推荐使用中文等大字符集
- 变量名称应具有物理意义
 - 变量名应为英文单词，下划线分隔，不缩写
 - 变量名不能使用拼音

变量的写入值

• 变量写入值的方式

方式	说明	格式	示例
赋值	利用赋值运算符改变左值的值	<变量名> = <表达式>	<code>var = 65;</code>
写内存	调用scanf和memset等函数写入内存改变值	<函数名> (<参数列表>)	<code>scanf("%f", &var);</code>

取地址操作符&

• 赋值

– 按优先级计算，用右值改变左值

• 写内存（以scanf函数为例）

- 按格式字符串解析用户输入的内容，赋值到后续地址列表
- 一般变量应取地址，字符串名不取地址

变量的读取值

- 变量读取值的方式

- 变量是表达式，可以和操作符组合组成表达式
- 变量除非作为左值，在表达式计算中，取值参与计算

第2步, $1700 * 0.56 = 952$

第1步, weight取值0.56

```
value = 1700.0 * weight * 14.5833;
```

第4步, value赋值为13883.3016

第3步, $952 * 14.5833 = 13883.3016$

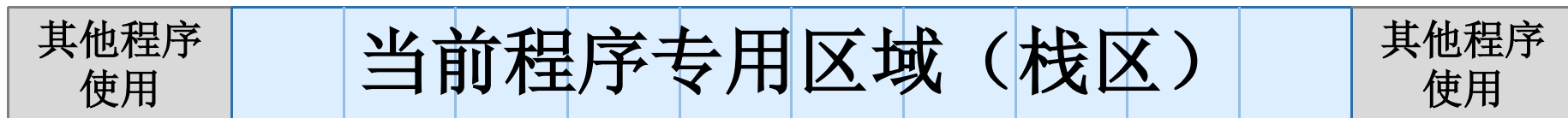
- 注意事项

- 所有的变量必须**先声明再赋值后使用**，否则结果不可控
- 变量的使用范围为声明语句后，作用域中使用

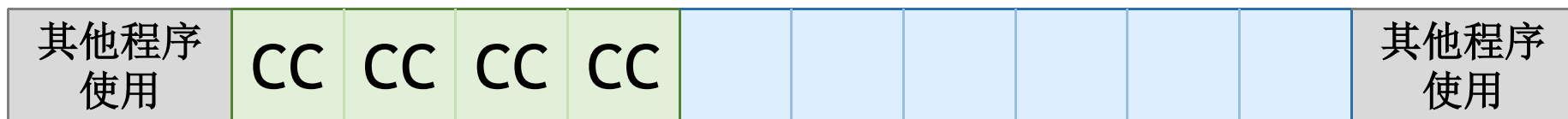
局部变量作用域在当前代码块，全局变量作用域在当前文件。具体规则，以后课程介绍

变量相关的内存变化

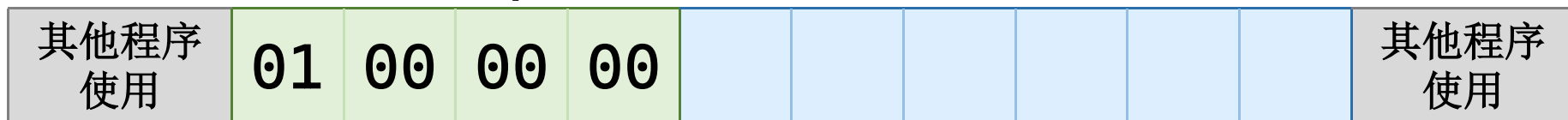
- 原始的内存空间



- 变量声明 (`int a;`) 时，开辟4字节空间并关联名称
a所在区域



- 变量赋值 (`a=1;`) 时，改变内存内容



- 变量求值 (`a`) 时，查询对应内存区域的内容（1）

常量

- 常量（Constants）：程序运行过程中保持值不变的量

- 例如：程序运行中 $\pi = 3.14$ 是不能随便改动的

- 常量的声明

```
const double PI = 3.14;
```

- 常量名一般不使用小写字母

- 常量具备变量的一切属性和方法

- 仅允许声明时赋值，此后常量名不得作为赋值表达式左值

修改常量产生编译错误

- 尝试修改const常量将产生编译错误

```
const float mult;  
mult = 23 * weight * 14.5833;
```

黄色底色的方框中是
错误的示例代码

- 尝试修改常量

- 编译时出现错误：为只读变量（常量）赋值

```
const.c: In function 'main':  
const.c:9:5: error: assignment of read-only variable 'mult'  
    mult = 23;  
    ^
```

绿色底色的方框中是
编译器的输出信息

- 对#define常量或数值常量赋值也将产生编译错误

- 宏是简单替换，不因宏定义而产生额外的空间

常量和宏

- 常量（Constants）：运行过程中保持值不变的量
- 用宏定义常量：为特殊意义常数取一个好记的名称
- 常量占用存储空间，宏和常数不开辟专门的存储空间

分类	示例	格式
常数	14.5833	整数、实数、字符、字符串，或者八进制、十六进制。
宏定义	<code>#define PP0 14.5833</code>	预编译指令 <code>#define</code> ，宏名，宏的展开
常量	<code>const float X = 1700.0;</code>	关键字 <code>const</code> ，数据类型，变量名，赋值符号，值。

预定义为#define的常量

- 格式 `#define NAME value // comments`
- 行为：预编译器替换所有关键字，再交给编译器编译
 - 文中除注释和字符串外所有完整的NAME都替换为value

使用宏定义的语句	宏展开的语句
<code>circum = 2.0 * PI * radius;</code>	<code>circum = 2.0 * 3.14159 * radius;</code>
<code>"PI is 3.14"</code>	<code>"PI is 3.14"</code>
<code>area = PIs * radius * radius;</code>	<code>area = PIs * radius * radius;</code>
宏定义	s = PI * r * r; 宏展开的语句
<code>#define PI 3.1415</code>	<code>s = 3.1415 * r * r;</code>
<code>#define PI 3 + 0.1415</code>	<code>s = 3 + 0.1415 * r * r;</code>
<code>#define PI = 3.1415</code>	<code>s = = 3.1415 * r * r;</code>

```

/* pizza.c -- uses defined constants in a pizza context */
#include <stdio.h>
#define PI 3.14159
int main(void)
{
    float area, circum, radius;

    printf("What is the radius of your pizza?\n");
    scanf("%f", &radius);
    area = PI * radius * radius;
    circum = 2.0 * PI * radius;
    printf("Your basic pizza parameters are as follows:\n");
    printf("circumference = %1.2f, area = %1.2f\n",
    circum, area);
    return 0;
}

```

有经验的程序员用宏定义来表示常数，明确常量的物理意义

What is the radius of your pizza?

35↵

Your basic pizza parameters are as follows:
circumference = 219.91, area = 3848.45

目录

	1	变量和常量
	2	常数的表示
	3	整型和浮点型
	4	相关的编程错误
	5	小结

整数常数的表示

- 格式（大小写无关） `[正负号]<十/八/十六进制常数>[整数后缀]`

正负号	可省略	示例（106）
正号（默认）	是	+106 或 106
负号	否	-106

进制序列	前缀	数字字符集	示例（106）
十进制	无	0123456789	106 ($=1*10^2+0*10^1+6*10^0$)
八进制	0	01234567	0152 ($=1*8^2+5*8^1+2*8^0$)
十六进制	0x	0123456789ABCDEF	0x6a ($=6*16^1+10*16^0$)

后缀类型	后缀	示例（106）
长型（默认）	l	106l
无符号	u	106u
64位	ll（vs里是i64）	106ll

字符常数的表示

- 字符常数（大小写有关）

'<字符序列>'

- 字符常数的类型是整型（4字节），而不是字符型
- 字符序列是指：除单引号（'）、反斜杠（\）或者换行符以外的所有源字符集成员，或者转义序列

转义序列举例	格式	示例
简单转义序列	反斜杠（\），非x小写字母或标点	\a \b \f \n \r \t \v \' \" \\ \?
八进制转义序列	反斜杠（\），1~3位八进制数（01234567）	\101
十六进制转义序列	反斜杠（\），小写字母x，十六进制数（0123456789ABCDEF）	\x41

整数常数的表示形式

• 整数（含字符）常数的主要表示形式

赋值方法	示例	值	特点
十进制写法	var=65;	65	不以0开头
八进制写法	var=0101;	65	以0开头，但不紧接x或X，后续数字是0~7。
十六进制写法	var=0x20;	32	以0x或0X开头，后续数字是0~9，A~F。
可见字符	var='A';	65	以单引号为界，中间只有一个字符
转义序列字符	var='\b';	9	以单引号为界，中间以反斜杠开头，跟着一个字母
八进制字符	var='\101';	65	以单引号为界，中间以反斜杠开头，但不紧接x或X，后续数字是0~7。
十六进制字符	var='\x20';	32	以单引号为界，中间以\x或\x开头，后续数字是0~9，A~F。

实数常数的表示

- 格式（大小写无关） `<正负号><小数常数>[指数][浮点后缀]`

小数常数序列	示例	值
数字序列+小数点+数字序列	30.106	30.106
小数点+数字序列	.106	0.106
数字序列+小数点	106.	106.0

指数部分序列	示例	值
e（大小写）+正负号（默认正）+数字序列	1.5E-2	0.015

浮点后缀	含义	示例（106）
l（默认）	double类型	-10.6l
f	float类型	10.6f

```
/* escape.c -- uses escape characters */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

Enter your desired monthly salary: \$■_____

光标在此闪烁

```
float salary;
```

打印到此

```
printf("\aEnter your desired monthly salary:"); /* 1 */
```

```
printf("_ $_____ \b\b\b\b\b\b\b\b"); /* 2 */
```

利用转义序列回退光标
(注意: OJ不支持)

```
scanf("%f", &salary);
```

```
printf("\n\t$%.2f a month is $%.2f a year.", salary,  
salary * 12.0); /* 3 */
```

```
printf("\rGee!\n"); /* 4 */
```

```
return 0;
```

Enter your desired monthly salary: \$123↓____

Gee! \$123.00 a month is \$1476.00 a year.

整数常数的原则

- 书写原则

- 一般不加前后缀，除非不加前后缀会出现错误

- 前缀： $-3 < -2$ （真）； $3 < 2$ （假）

- 后缀： $65536\text{ul} * 65536\text{ul} > 0$ （真）； $65536 * 65536 > 0$ （假）

- 尊重书写习惯

- 小数点前的0不建议省略

- 简单的小数直接写，前导0多的小数应用科学计数法

- 幂数标记 e/E 大小写都可以，但是应全篇统一

目录

	3	整型和浮点型
	3.1	整型
	3.2	浮点型
	3.3	类型的选用
	4	相关的编程错误

整型分类

- 整型类型（按符号划分）
 - 有符号（signed，默认）、无符号（unsigned）
- 整型类型（按长度划分）
 - 字符型（char）：8位
 - 短整型（short，16位机器int）：16位
 - 长整型（long，32和64位机器int）：32位
 - 超长整型（long long）：64位
 - 注意：在Linux GCC 64位编译器下，long被视为超长整型

字符型可以视为取值范围很小的整型

整型分类（32/64位架构）

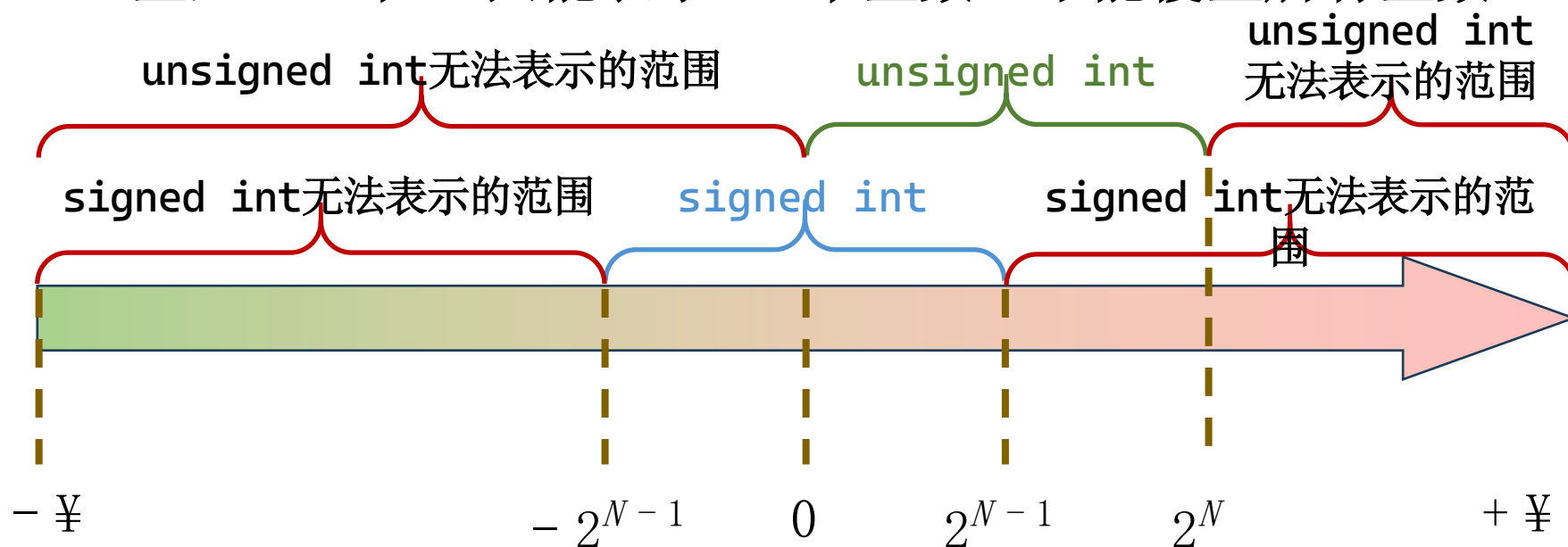
- 在32或64位架构下整型所有分类
 - `signed` 和 `int` 单词可省略不写（但不能略至没有词）

名称	有符号类型	无符号版本
字符型 ($n = 3$)	<code>signed char; char</code>	<code>unsigned char</code>
短整型 ($n = 4$)	<code>signed short int; signed short; short int; short</code>	<code>unsigned short int; unsigned short</code>
长整型 ($n = 5$)	<code>signed long int; signed long; signed int; long int; long; int; signed</code>	<code>unsigned long int; unsigned int; unsigned; unsigned long;</code>
超长整型 ($n = 6$)	<code>signed long long int; signed long long; long long int; long long</code>	<code>unsigned long long int; unsigned long long</code>

整数的精度和范围

- 整型的精度是 1
- 整型不是整数

– 整型 (N 位) 只能表示 2^N 个整数, 不能覆盖所有整数



说明: 图示以 $N = 32$ 为例。

整数的精度和范围

- 整数内存占用: $N = 2^n$ bit
- 整数范围: 有符号 $[-2^{N-1}, 2^{N-1})$, 无符号 $[0, 2^N)$

名称	长度 ($N=2^n$)	有符号取值范围 ($[-2^{N-1}, 2^{N-1})$, 精度为1)	无符号取值范围 ($[0, 2^N)$, 精度为1)
字符型 ($n = 3$)	8 b	-128 ~ +127	0 ~ 255
短整型 ($n = 4$)	16 b	-32,768 ~ +32,767	0 ~ 65,535
长整型 ($n = 5$)	32 b	-2,147,483,648 ~ +2,147,483,647	0 ~ +4,294,967,295 (4×10^9)
超长整型 ($n = 6$)	64 b	-9,223,372,036,854,775,808 ~ +9,223,372,036,854,775,807	0 ~ +18,446,744,073,709,551,615 (1.8×10^{19})

整数的内存格式（字符型为例）

- 字符（character）

位置	7	6	5	4	3	2	1	0	值
二进制	1	0	1	1	0	0	1	0	-
十六进制	B				2				-
unsigned char	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	178
signed char	—	$256 - 178 = 78$							-78

位、字节和字

- 位（bit）：二元不确定性
 - 有2种情况概率相等，从未知到已知所获得的信息量为1位
 - 有 2^N 种情况概率相等，从未知到已知所获得的信息量为 N 位
- 字节（byte）：内存处理数据的最小单位
 - 1 byte = 8 bits
- 字（word）：用于一次性处理事务的固定长度位组
 - 现代计算机的字长通常为32、64位
 - 字长指寄存器的大小（即指示内存的范围）

进制：看信息的角度

$$y_i = \frac{X}{M^i} \bmod M$$

9	0	2	7
9000	0	20	7

$$9 \cdot 10^3 + 0 \cdot 10^2 + 2 \cdot 10^1 + 7 \cdot 10^0$$

2	1	5	0	3
8192	512	320	0	3

$$2 \cdot 8^4 + 1 \cdot 8^3 + 5 \cdot 8^2 + 0 \cdot 8^1 + 3 \cdot 8^0$$

例如：将10进制的9027转换为8进制数

$$9 \cdot 10^3 + 0 \cdot 10^2 + 2 \cdot 10^1 + 7 \cdot 10^0 = 9027$$

$$9027 \div 8 = 1128 \text{ 余 } 3$$

$$1128 \div 8 = 141 \text{ 余 } 0$$

$$141 \div 8 = 17 \text{ 余 } 5$$

$$17 \div 8 = 2 \text{ 余 } 1$$

$$2 \div 8 = 0 \text{ 余 } 2$$

答案： $(9027)_{10} = (21503)_8$ 。

进制：不同进制的转换

- 不同进制与二进制的转换

数字	BIT 3	BIT 2	BIT 1	BIT 0	数字	BIT 3	BIT 2	BIT 1	BIT 0
0	0	0	0	0	8	1	0	0	0
1	0	0	0	1	9	1	0	0	1
2	0	0	1	0	10 (A)	1	0	1	0
3	0	0	1	1	11 (B)	1	0	1	1
4	0	1	0	0	12 (C)	1	1	0	0
5	0	1	0	1	13 (D)	1	1	0	1
6	0	1	1	0	14 (E)	1	1	1	0
7	0	1	1	1	15 (F)	1	1	1	1

ASCII码表

- 字符型数据的值是8位（bit）整数
 - 其ASCII码所对应的字符是输入输出格式之一
- 表示（初学者只需知道它们是有序的）
 - 0-31：非打印控制字符，用于控制打印机等外围设备。
 - 例如，12代表换页/新页功能，指示打印机跳到下一页的开头。
 - 32-127：可打印字符，能在键盘上找到的字符。
 - 数字127代表 DELETE 命令。
 - 128-255：扩展ASCII打印字符

ASCII码表

扩展内容

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
0_	NUL 0000 0	SOH 0001 1	STX 0002 2	ETX 0003 3	EOT 0004 4	ENQ 0005 5	ACK 0006 6	BEL 0007 7	BS 0008 8	HT 0009 9	LF 000A 10	VT 000B 11	FF 000C 12	CR 000D 13	SO 000E 14	SI 000F 15
1_	DLE 0010 16	DC1 0011 17	DC2 0012 18	DC3 0013 19	DC4 0014 20	NAK 0015 21	SYN 0016 22	ETB 0017 23	CAN 0018 24	EM 0019 25	SUB 001A 26	ESC 001B 27	FS 001C 28	GS 001D 29	RS 001E 30	US 001F 31
2_	SP 0020 32	! 0021 33	" 0022 34	# 0023 35	\$ 0024 36	% 0025 37	& 0026 38	' 0027 39	(0028 40	) 0029 41	* 002A 42	+ 002B 43	, 002C 44	- 002D 45	. 002E 46	/ 002F 47
3_	0 0030 48	1 0031 49	2 0032 50	3 0033 51	4 0034 52	5 0035 53	6 0036 54	7 0037 55	8 0038 56	9 0039 57	: 003A 58	; 003B 59	< 003C 60	= 003D 61	> 003E 62	? 003F 63
4_	@ 0040 64	A 0041 65	B 0042 66	C 0043 67	D 0044 68	E 0045 69	F 0046 70	G 0047 71	H 0048 72	I 0049 73	J 004A 74	K 004B 75	L 004C 76	M 004D 77	N 004E 78	O 004F 79
5_	P 0050 80	Q 0051 81	R 0052 82	S 0053 83	T 0054 84	U 0055 85	V 0056 86	W 0057 87	X 0058 88	Y 0059 89	Z 005A 90	[005B 91	\ 005C 92	] 005D 93	^ 005E 94	_ 005F 95
6_	` 0060 96	a 0061 97	b 0062 98	c 0063 99	d 0064 100	e 0065 101	f 0066 102	g 0067 103	h 0068 104	i 0069 105	j 006A 106	k 006B 107	l 006C 108	m 006D 109	n 006E 110	o 006F 111
7_	p 0070 112	q 0071 113	r 0072 114	s 0073 115	t 0074 116	u 0075 117	v 0076 118	w 0077 119	x 0078 120	y 0079 121	z 007A 122	{ 007B 123	 007C 124	} 007D 125	~ 007E 126	DEL 007F 127

目录

	3	整型和浮点型
	3.1	整型
	3.2	浮点型
	3.3	类型的选用
	4	相关的编程错误

浮点型分类

- 浮点型都是有符号的类型
- 浮点型类型（按长度划分）

$$\pm \text{尾数} * 2^{\text{指数}}$$

小数范围：
 $2^{23}=8388608$

关键字	名称	二进制位数				十进制位数	
		总	符号	指数	小数	小数精度	指数范围
float	单精度	32	1	8	23	-6 ~ -7	-37 ~ +38
double	双精度	64	1	11	52	-15 ~ -16	-307 ~ +308
long double	长双精度	80	1	15	64	-19 ~ -20	-4391 ~ +4392

* long double 为了对齐实际是12B或16B，不足补0。（初学者不需要掌握）

指数范围：

$$2^8=256$$

有符号型，所以-127~128（实际上去掉正负无穷-126~127）
- 2^{128} 到 2^{128} ，约等于-3.4E38 到 +3.4E38

浮点型的内存格式

- 单精度浮点型 (float)

- 浮点型的存储一般使用二进制的科学计数法。

符号	指数								分数																						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
1	0	1	1	1	1	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
—	—	0	0	0	0	0	2	1	0	$\frac{1}{4}$	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
—	2^{-3}								$1 + 2^{-2}$																						
-1*（1.0+0.25）*0.125=-0.15625																															

- 符号位 (S)：1，表示负数。
- 指数位 (E)：01111100，二进制值为 124。真实指数计算为：124 - 127 = -3，即 2^{-3} 。
- 尾数：小数部分是 0000000000000000000010，只考虑有效部分，二进制值为 1/4 (即 0.25)。

双精度浮点型的内存格式

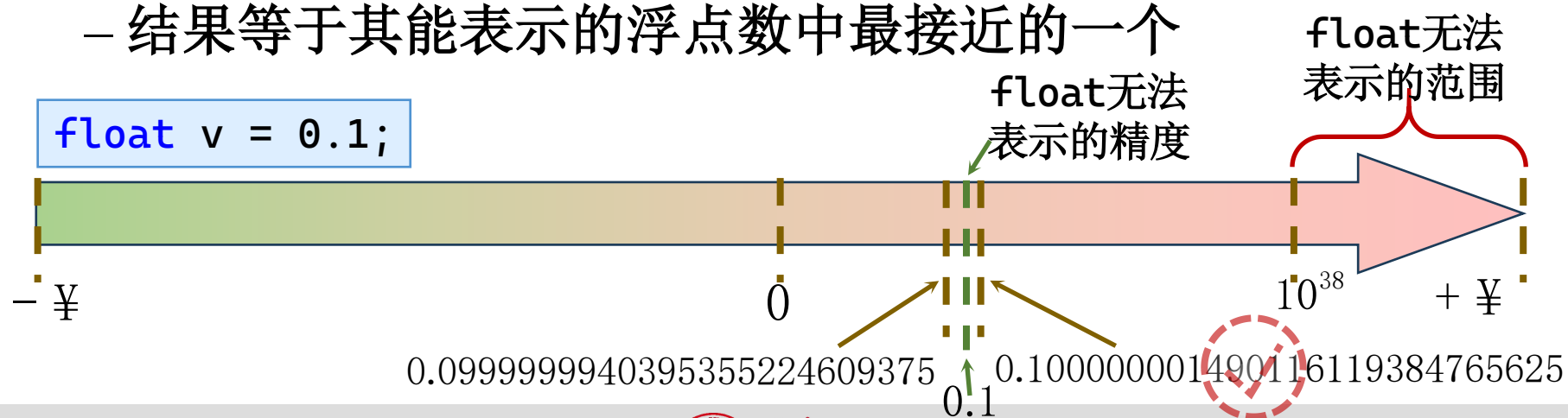
- 双精度浮点型（double）

- 双精度分数部分使用更多位，比单精度浮点型精确

符号	指数								分数																							
63	62	61	60	..	55	54	53	52	51	50	49	48	47	46	..	35	34	33	32	..	25	24	..	07	06	05	04	03	02	01	00	
1	0	1	1	..	1	1	0	0	0	1	0	0	0	0	..	0	0	0	0	..	0	0	..	0	0	0	0	0	0	0	0	
—	—	0	0	..	0	0	2	1	0	$\frac{1}{4}$	0	0	0	0	..	0	0	0	0	..	0	0	..	0	0	0	0	0	0	0	0	
—	2^{-3}								$1 + 2^{-2}$																							
-1*（1.0+0.25）*0.125=-0.15625																																

浮点型的精度

- 浮点型不是实数
 - 实数的范围是无穷大，精度也是无穷精度
 - 浮点型（ N 位）只能表示 2^N 个实数，不能覆盖所有实数
 - 浮点型表示范围由指数部分决定，精度由小数部分决定
- 浮点型变量常用有理数赋值，结果可能不如预期
 - 结果等于其能表示的浮点数中最接近的一个



浮点型的精度问题

- 设 `float PI=3.14, r=1.5, h=3;`
 - 则PI值为3.1400001049041, r和h值为原值
 - 不同的乘法顺序得到不同的结果（失之毫厘谬以千里）

表达式	计算次序	打印为%.10lf	打印为%.2lf
<code>r*r*pi*h</code>	2.25 → 7.0650000572	21.1949996948	21.19
<code>r*r*h*pi</code>	2.25 → 6.75	21.1950016022	21.20

- 正确的做法

- 使用精确的数据类型
- 先乘大数再乘小数

```
double PI=3.14, r=1.5, h=3;
```

```
PI * h * r * r
```

```
/* floater.c -- demonstrates round-off error */
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    float a, b;
```

```
    b = 2.0e20 + 1.0;
```

```
    a = b - 2.0e20;
```

```
    printf("%f \n", a);
```

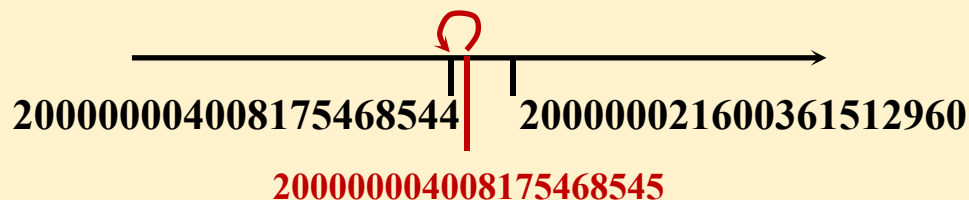
```
    printf("%e \n", a);
```

```
    return 0;
```

```
}
```

2e20最接近的是200000004008175468544，该数加1为200000004008175468545，该数赋值给float类型变量b时，仍回到200000004008175468544。

b是200000004008175468544，该数减去2e20，得到4008175468544，该数赋值给float类型变量a时，最近的是4008175468544。



4008175468544.000000

4.008175e+012

```

/* typesize.c -- prints out type sizes */
#include <stdio.h>
int main(void) {
    /* c99 provides a %zd specifier for sizes */
    printf("Type int has a size of %zd bytes.\n", sizeof(int));
    printf("Type char has a size of %zd bytes.\n", sizeof(char));
    printf("Type long has a size of %zd bytes.\n", sizeof(long));
    printf("Type long long has a size of %zd bytes.\n",
sizeof(long long));
    printf("Type double has a size of %zd bytes.\n",
sizeof(double));
    printf("Type long double has a size of %zd bytes.\n",
        sizeof(long double));
    return 0;
}

```

```

Type int has a size of 4 bytes.
Type char has a size of 1 bytes.
Type long has a size of 4 bytes.
Type long long has a size of 8 bytes.
Type double has a size of 8 bytes.
Type long double has a size of 12 bytes.

```

目录

	3	整型和浮点型
	3.1	整型
	3.2	浮点型
	3.3	类型的选用
	4	相关的编程错误

基本数据类型的使用原则

- 声明少量的变量时，不折腾
 - 优先整数用int，字符用char，实数用double
- 默认情况无法满足要求时，按精度和值域选用
 - 整型int无法表示的整数用long long int
 - 双精度小数double无法表示的实数用long double
- 声明规模较大的数组，尽量选择更小的类型
 - 因为大量的变量使用太大的数据类型，会影响内存占用


```

/* print2.c - more printf() properties */
#include <stdio.h>
int main(void)
{
    unsigned int un = 3000000000; /* system with 32-bit int */
    short end = 200;                /* and 16-bit short */
    long big = 65537;
    long long verybig = 12345678908642;

    printf("un = %u and not %d\n", un, un);
    printf("end = %hd and %d\n", end, end);
    printf("big = %ld and not %hd\n", big, big);
    printf("verybig= %lld and not %ld\n", verybig, verybig);

    return 0;
}

```

un = 3000000000 and not -1294967296
 end = 200 and 200
 big = 65537 and not 1
 verybig= 12345678908642 and not 1942899938

```

/* charcode.c - displays code number for a character */
#include <stdio.h>
int main(void)
{
    char ch;

    printf("Please enter a character.\n");
    scanf("%c", &ch); /* user inputs character */
    printf("The code for %c is %d.\n", ch, ch);

    return 0;
}

```

ch的值（67）在%c下以ASCII码字符显示；
在%d下以十进制显示

输入以后需要按回车键，才能将缓冲的内容发送给程序

Please enter a character.

C↵

The code for C is 67.

目录

1	变量和常量
2	常数的表示
3	整型和浮点型
4	相关的编程错误
5	小结

相关的编程错误

- 学生应意识到
 - 逐一记诵错误之处，从而完全避免错误，是绝无可能的
 - 发现错误依赖系统的测试和调试方法
 - 了解常见的编程错误，可以加快找到错误

变量未赋值即使用

- 变量未赋值即使用，结果将不可控
 - 尤其自增减等操作符，需要先读取变量

```
char c = 0;  
c += 3;  
printf("%c", c);
```

```
char c;  
c += 3;  
printf("%c", c);
```

- 使用scanf可能未解析成功，因而应判断返回值

```
char c;  
if (scanf("%c", &c) == 1)  
    printf("%c", c);
```

```
char c;  
scanf("%c", &c);  
printf("%c", c);
```

变量未赋值即使用

- 在代码块（如：循环体）内应先赋值再使用
 - 变量应声明在其使用的最小范围内
 - 除非将该声明语句移至其下代码块会无法得到正确结果
 - 变量应在代码块最前方赋初始值，不应在使用后赋初始值

```
int main() {  
    int n = 0;  
    while (scanf("%d", &n) == 1)  
{  
        int i, b = 1;  
        for (i = 1; i <= n; i++)  
            b = b * i;  
    }  
}
```

```
int main() {  
    int n = 0, i, b = 1;  
    while (scanf("%d", &n) == 1)  
{  
        for (i = 1; i <= n; i++)  
            b = b * i;  
        b = 1;  
    }  
}
```

混淆数字字符和数字的概念

- 数字字符 '0' — '9' 和数值 0 — 9 不同
- 不同的格式说明符在键盘按下  时，解析的值不同

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	...
0_	NUL 0000 0	SOH 0001 1	STX 0002 2	ETX 0003 3	EOT 0004 4	ENQ 0005 5	ACK 0006 6	BEL 0007 7	BS 0008 8	HT 0009 9	...
...	字符 '0' — '9', 数值 48 — 57...										...
3_	0 0030 48	1 0031 49	2 0032 50	3 0033 51	4 0034 52	5 0035 53	6 0036 54	7 0037 55	8 0038 56	9 0039 57	...

说明符	数值	对应字符
%c	48	'0'
%d	0	'\0'

```
char c = 0;
scanf("%c", &c);
if (c == 0)
    return 0;
```

```
char c = 0;
scanf("%c", &c);
if (c == '0')
    return 0;
```

```
char c = 0;
scanf("%d", &c);
if (c == 0)
    return 0;
```

赋值时类型不匹配

- 赋值时，右值类型不匹配左值，编译器报告警

warning: overflow in implicit constant conversion [-Woverflow]

错误类型	示例代码片段	运行功能
用过大的数赋值过小的类型	<code>char num = 259;</code>	变量num的值为3。
	<code>char num = 385;</code>	变量num的值为-127。
用实数赋值整型	<code>char num = 259.3;</code>	变量值为127。
	<code>char num = -385.3;</code>	变量值为-128。
用有符号的数给无符号的变量赋值	<code>unsigned char num = -2;</code>	变量值为254。
	<code>unsigned char num = 2.5;</code>	变量值为0。
	<code>unsigned int a = 3; int res = (a - 4) > 0;</code>	变量res的值为1。
有符号写成无符号造成运算结果溢出	<code>int num = 65535; int res = (num*num > 0);</code>	变量res的值为0。

整数类型的溢出

- 整数类型的取值范围（如图）

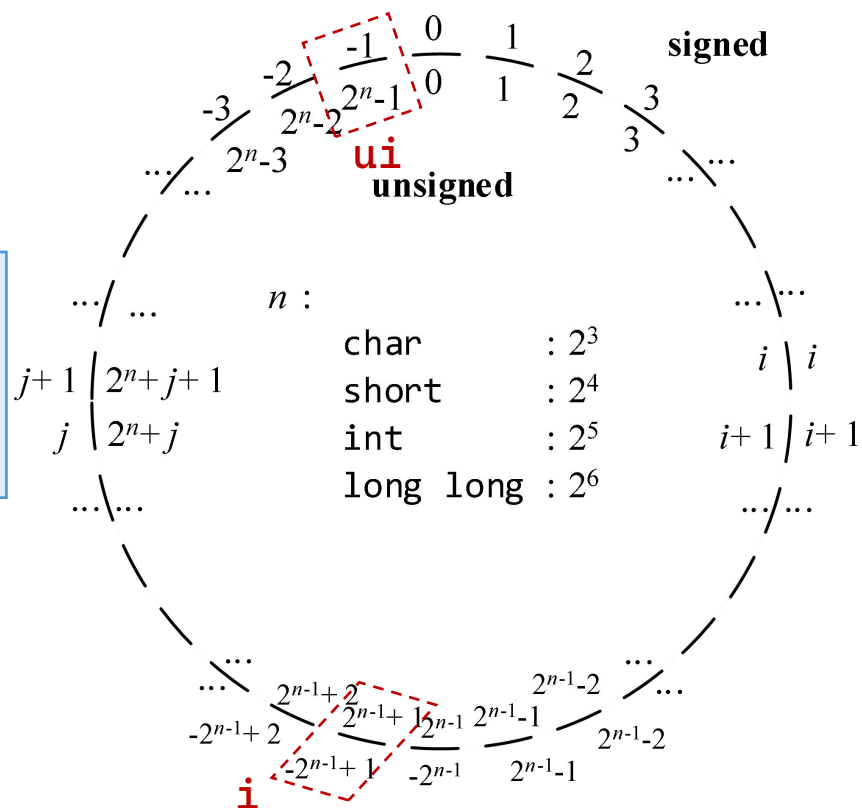
- 溢出：表达式超过了其数据类型的表示范围

- 溢出后的表现

- 根据图示自动转换

```
unsigned int ui = -1; 225 - 1  
int i = 2147483649u; 225-1 + 1  
printf("%u\n", i); // -2147483647
```

- 程序员应该避免溢出的发生



```

/* toobig.c-exceeds maximum int size on our system */
#include <stdio.h>
int main(void)
{
    int i = 2147483647;
    unsigned int j = 4294967295;

    printf("%d %d %d\n", i, i+1, i+2);
    printf("%u %u %u\n", j, j+1, j+2);

    return 0;
}

```

```

print1.c: In function 'main':
print1.c:6:5: warning: this decimal constant is
unsigned only in ISO C90 [enabled by default]
    unsigned int j = 4294967295;
    ^

```

```

2147483647 -2147483648 -2147483647
4294967295 0 1

```

忽视浮点型精度问题的错误

- 浮点型存在精度问题

- 运算结果应为 x 时，实际结果可能是 y

- 原因：十进制小数转换为二进制需要更多的有限数字表示

- 浮点型比较大小关系，会以一定概率出现答案错误

```
( 1.1f - 1.f - .1f ) == 0 // 1.1000000238
```

```
fabs( 1.1f / 9 * 3 * 3 == 1.1f ) < 1e-6
```

$$1.099999904632568359375 = 1 \times 2^0 + 1 \times 2^{-4} + 1 \times 2^{-5} + 1 \times 2^{-8} + 1 \times 2^{-9} + 1 \times 2^{-12} \\ + 1 \times 2^{-13} + 1 \times 2^{-16} + 1 \times 2^{-17} + 1 \times 2^{-20} + 1 \times 2^{-21}$$

$$1.10000002384185791015625 = 1 \times 2^0 + 1 \times 2^{-4} + 1 \times 2^{-5} + 1 \times 2^{-8} + 1 \times 2^{-9} + 1 \times 2^{-12} \\ + 1 \times 2^{-13} + 1 \times 2^{-16} + 1 \times 2^{-17} + 1 \times 2^{-20} + 1 \times 2^{-21} + 1 \times 2^{-23}$$

$$0.0999999940395355224609375, 0.100000001490116119384765625$$

忽视浮点型精度问题的错误

- 浮点型上下和舍入取整，会以一定概率出现答案错误
 - 切勿认为double类型就不会有精度问题 $\sqrt[3]{125} = 5$

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main() {
```

```
    double x = 125;
```

```
    double y = 1. / 3;
```

```
    double d = pow(x, y);
```

```
    int u = d;
```

```
    printf("Pow = %.16lf\n", d);
```

```
    printf("Floor Pow = %.16lf\n", floor(d));
```

```
    printf("Floor Pow Int = %d\n", u);
```

```
    return 0;
```

```
}
```

```
Pow = 4.9999999999999991
```

```
Floor Pow = 4.0000000000000000
```

```
Floor Pow Int = 4
```

数学库函数存在精度问题，很难想象。但解决这一问题的有效方法，还是在于认识全面测试程序的重要性。在浮点型转换为整数时，多留个心眼。

布尔类型

- `_Bool` 类型 (`Boolean`)
 - 在C99引入，用于表示布尔值
 - `true` (非0)、`false` (0)
 - 大小: 1 bit
 - 其实是用`int`实现。

可移植类型

- 可移植类型应包含 `stdint.h` 和 `inttypes.h`
- `int`等对不同架构大小不同，需要“确切长度类型”
 - `int8_t` `int16_t` `int32_t` ...
- 最小长度类型：能容纳指定长度的最小类型
 - 如： `int_least8_t`
- 最快最小长度类型：使计算达到最快的最小长度类型
 - 如： `int_fast8_t`

目录

	1	变量和常量
	2	常数的表示
	3	整型和浮点型
	4	相关的编程错误
	5	小结

谢谢观看

